Paquetes en PL/SQL

Organizar, encapsular y reutilizar la lógica en la base de datos

Bases de Datos · Ingeniería de Sistemas · Oracle Database 19c

Paquetes

Bases de Datos · Ingeniería de Sistemas · PL/SQL

Dónde estamos

- Ya escribimos procedimientos y funciones, y sabemos atender sus excepciones. Con decenas de subprogramas sueltos en el esquema, falta orden: qué pertenece a qué módulo y qué puede usar cualquiera.
- Un esquema con cien procedimientos sueltos no es un sistema: es un cajón.

Recorrido

- Qué es un paquete: especificación y cuerpo
- Público y privado, estado de sesión e inicialización
- Sobrecarga, excepciones y constantes compartidas
- Dependencias, diccionario de datos, paquetes de Oracle y antipatrones

Qué es un paquete

Un módulo con contrato público e implementación oculta

- Agrupa tipos, constantes, variables, cursores, excepciones y subprogramas relacionados.
- Se compila y se almacena en la base de datos como un objeto de dos partes: especificación y cuerpo.
- Quien lo usa solo conoce la especificación; el cuerpo puede cambiar sin afectarlo.
- La primera referencia en una sesión carga su código en memoria y crea su estado para esa sesión.

Un paquete es a PL/SQL lo que una clase con miembros públicos y privados es a Java, sin herencia ni instancias.

Especificación y cuerpo

ESPECIFICACIÓN (PACKAGE)

- Declara todo lo que es público
- Es el contrato con quien lo usa
- Puede existir sin cuerpo si solo tiene tipos, constantes o excepciones
- Si cambia, puede invalidar a los dependientes

CUERPO (PACKAGE BODY)

- Implementa lo declarado en la especificación
- Puede tener elementos privados propios
- Se compila después de la especificación
- Si cambia, los dependientes siguen válidos

Se puede compilar la especificación y escribir los programas que la usan antes de tener listo el cuerpo.

La especificación

Bases de Datos · Ingeniería de Sistemas · PL/SQL

Solo declaraciones: aquí no hay código ejecutable.

```
CREATE OR REPLACE PACKAGE pkg_inventario IS
    -- constantes publicas
    c_max_movimiento CONSTANT PLS_INTEGER := 500;

    -- catalogo de errores del modulo
    e_stock_insuficiente EXCEPTION;
    PRAGMA EXCEPTION_INIT(e_stock_insuficiente, -20101);

    -- subprogramas publicos: solo la firma
    FUNCTION  fn_stock_disponible(p_id_producto NUMBER, p_id_bodega NUMBER) RETURN NUMBER;
    PROCEDURE sp_ingresar (p_id_producto NUMBER, p_id_bodega NUMBER, p_cantidad NUMBER);
    PROCEDURE sp_despachar(p_id_producto NUMBER, p_id_bodega NUMBER, p_cantidad NUMBER);
END pkg_inventario;
/
```

Todo lo declarado aquí es visible para cualquier usuario con privilegio EXECUTE sobre el paquete. Cada elemento público es un compromiso.

El cuerpo


```
CREATE OR REPLACE PACKAGE BODY pkg_inventario IS
    g_operaciones PLS_INTEGER := 0;                     -- privado: estado

    PROCEDURE validar_cantidad(p_cantidad NUMBER) IS    -- privado
    BEGIN
        IF NVL(p_cantidad, 0) <= 0 OR p_cantidad > c_max_movimiento THEN
            RAISE_APPLICATION_ERROR(-20102, 'Cantidad invalida.');
        END IF;
    END validar_cantidad;

    PROCEDURE sp_despachar(p_id_producto NUMBER, p_id_bodega NUMBER,
                           p_cantidad NUMBER) IS        -- publico
    BEGIN
        validar_cantidad(p_cantidad);
        UPDATE inventario SET cantidad = cantidad - p_cantidad
         WHERE id_producto = p_id_producto AND id_bodega = p_id_bodega
           AND cantidad >= p_cantidad;
        IF SQL%ROWCOUNT = 0 THEN
            RAISE_APPLICATION_ERROR(-20101, 'Stock insuficiente.');
        END IF;
        g_operaciones := g_operaciones + 1;
    END sp_despachar;
    -- ... fn_stock_disponible y sp_ingresar
END pkg_inventario;
```

La firma en el cuerpo debe coincidir con la de la especificación. Un privado debe declararse antes de usarse dentro del cuerpo.

Público, privado y local

Alcance	Dónde se declara	Desde dónde se ve	Ejemplo
Público	Especificación	Cualquier usuario con EXECUTE	sp_despachar
Privado	Solo en el cuerpo	Solo dentro del cuerpo del paquete	validar_cantidad
Local	Dentro de un subprograma	Solo dentro de ese subprograma	v_disponible

- Exponga lo mínimo: un elemento público no se puede retirar sin romper a quien lo usa.
- Validaciones, utilidades y variables de estado casi siempre van privadas.
- Invocar un privado desde afuera no compila: PLS-00302, el componente debe ser declarado.

Estado del paquete

Las variables declaradas a nivel de paquete viven lo que dura la sesión.

```
-- Sesion A
BEGIN
    pkg_inventario.sp_despachar(1, 1, 30);
    DBMS_OUTPUT.PUT_LINE(pkg_inventario.fn_operaciones_sesion);   -- 1
END;
/
BEGIN
    DBMS_OUTPUT.PUT_LINE(pkg_inventario.fn_operaciones_sesion);   -- sigue en 1
END;
/
-- Sesion B: tiene su propia copia, empieza en 0
```

- Cada sesión tiene su copia: el estado no se comparte entre usuarios ni conexiones.
- No es transaccional: ROLLBACK no devuelve una variable de paquete a su valor anterior.
- Si se recompila el cuerpo, la sesión pierde su estado y recibe ORA-04068 en la siguiente llamada.

Con pools de conexiones, la sesión la hereda el siguiente que tome la conexión: no guarde allí datos de un usuario.

Inicialización del paquete

Un bloque BEGIN al final del cuerpo que corre una sola vez por sesión.

```
CREATE OR REPLACE PACKAGE BODY pkg_inventario IS
    g_fecha_carga DATE;
    g_iva         NUMBER;
    -- ... subprogramas ...
BEGIN
    -- se ejecuta en la primera referencia al paquete dentro de la sesion
    g_fecha_carga := SYSDATE;
    SELECT valor INTO g_iva FROM parametro WHERE nombre = 'IVA';
END pkg_inventario;
```

- Sirve para cargar parámetros o tablas de referencia que se consultan muchas veces.
- Si la inicialización falla, la llamada que la disparó recibe ese error.
- Manténgala corta: cada sesión nueva paga su costo en la primera llamada.

Sobrecarga

Varios subprogramas con el mismo nombre y distinta lista de parámetros.

```
FUNCTION fn_stock_disponible(p_id_producto NUMBER, p_id_bodega NUMBER) RETURN NUMBER;
FUNCTION fn_stock_disponible(p_id_producto NUMBER) RETURN NUMBER;   -- todas las bodegas

v_bodega := pkg_inventario.fn_stock_disponible(1, 1);   -- primera version
v_total  := pkg_inventario.fn_stock_disponible(1);      -- segunda version
```

Sí permite sobrecargar	No basta con que cambie
El número de parámetros	Solo el tipo de retorno
La familia del tipo: NUMBER, VARCHAR2, DATE	Subtipos de la misma familia: NUMBER e INTEGER
El orden de los tipos	Solo el nombre del parámetro o su modo IN / OUT

La sobrecarga solo existe en paquetes y subprogramas locales: dos procedimientos sueltos no pueden llamarse igual.

Excepciones y constantes compartidas

Bases de Datos · Ingeniería de Sistemas · PL/SQL

La especificación como catálogo del módulo.

```
-- En la especificacion
e_stock_insuficiente EXCEPTION;
PRAGMA EXCEPTION_INIT(e_stock_insuficiente, -20101);

-- En el cuerpo: se levanta con numero y mensaje
RAISE_APPLICATION_ERROR(-20101, 'Stock insuficiente en la bodega ' || p_id_bodega);

-- En cualquier llamador: se atiende por nombre, no por numero
BEGIN
    pkg_inventario.sp_despachar(2, 1, 50);
EXCEPTION
    WHEN pkg_inventario.e_stock_insuficiente THEN
        DBMS_OUTPUT.PUT_LINE(SQLERRM);
END;
```

Une las dos piezas de la clase anterior: RAISE_APPLICATION_ERROR lleva número y mensaje al cliente, y PRAGMA EXCEPTION_INIT le da un nombre que cualquier llamador puede atrapar.

Dependencias y recompilación

Cambio	Efecto en quienes usan el paquete
Solo el cuerpo	Siguen VALID; las sesiones con estado reciben ORA-04068 una vez
La especificación	Pueden quedar INVALID; se recompilan solos en la siguiente llamada
DROP PACKAGE BODY	La especificación sigue; invocar un subprograma da ORA-04067
DROP PACKAGE	Elimina especificación y cuerpo; los dependientes quedan INVALID

```
ALTER PACKAGE pkg_inventario COMPILE;        -- especificacion y cuerpo
ALTER PACKAGE pkg_inventario COMPILE BODY;   -- solo el cuerpo
```

La regla de oro: especificación estable, cuerpo libre para evolucionar.

Cómo se invoca un paquete


```
-- Desde PL/SQL, con notacion por nombre
pkg_inventario.sp_despachar(p_id_producto => 1, p_id_bodega => 1, p_cantidad => 30);

-- Desde SQL*Plus o SQL Developer
EXEC pkg_inventario.sp_ingresar(3, 2, 25)

-- Desde SQL: solo funciones publicas sin DML
SELECT codigo, pkg_inventario.fn_stock_disponible(id_producto) AS total FROM producto;

-- Desde otro esquema
GRANT EXECUTE ON pkg_inventario TO app_ventas;
```

- Se referencia como paquete.elemento; desde otro esquema, esquema.paquete.elemento.
- Una función que hace DML no se puede usar dentro de un SELECT: ORA-14551.
- EXECUTE se otorga sobre el paquete completo, no por subprograma.

Por defecto el paquete corre con los privilegios de su dueño (AUTHID DEFINER): quien lo invoca no necesita permisos sobre las tablas.

El paquete en el diccionario de datos

Vista	Qué muestra
USER_OBJECTS	Estado VALID o INVALID de PACKAGE y PACKAGE BODY
USER_SOURCE	Código fuente línea por línea, filtrando por TYPE
USER_ERRORS	Errores de compilación con línea y posición
USER_PROCEDURES	Subprogramas públicos y su número de sobrecarga
USER_ARGUMENTS	Parámetros de cada subprograma, separados por OVERLOAD
USER_DEPENDENCIES	Quién depende del paquete y de qué depende él

SHOW ERRORS PACKAGE BODY pkg_inventario es el atajo de SQL*Plus y SQL Developer para consultar USER_ERRORS.

Paquetes que ya trae Oracle

Paquete	Para qué sirve
DBMS_OUTPUT	Mensajes hacia la consola del cliente
DBMS_UTILITY	FORMAT_ERROR_BACKTRACE, FORMAT_ERROR_STACK y otras utilidades
DBMS_SQL	SQL dinámico cuando no se conocen las columnas de antemano
DBMS_SCHEDULER	Programar trabajos dentro de la base de datos
DBMS_LOB	Leer y modificar CLOB y BLOB
DBMS_RANDOM	Valores aleatorios para generar datos de prueba
UTL_FILE	Leer y escribir archivos en el servidor de base de datos

DBMS_OUTPUT.PUT_LINE es la llamada a un procedimiento de paquete: lo usamos desde la primera clase.

Antipatrones

Bases de Datos · Ingeniería de Sistemas · PL/SQL

Antipatrón	Por qué es un problema
Todo público en la especificación	No hay encapsulamiento: cualquier ajuste interno rompe a alguien
Un paquete de utilidades con todo adentro	Un cambio en cualquier parte descarta el estado de todas las sesiones
Variables públicas modificables	Cualquier llamador altera el estado sin control ni validación
COMMIT dentro de los subprogramas	El llamador pierde el control de su transacción
Datos de usuario en el estado con pool	El siguiente que toma la conexión hereda esos valores
Tocar la especificación por cada ajuste	Invalida dependientes y fuerza recompilaciones en cascada

Buenas prácticas

- Un paquete por módulo de negocio, con un nombre que lo diga: pkg_inventario, pkg_facturacion.
- Especificación mínima y estable; validaciones y lógica auxiliar, privadas en el cuerpo.
- Declare en la especificación el catálogo de excepciones y constantes del módulo.
- Exponga el estado con funciones de consulta, nunca como variable pública.
- Use notación por nombre en llamadas con varios parámetros: p_cantidad => 30.
- Mantenga un script de pruebas por paquete y ejecútelo después de cada cambio.

Para retener

- Un paquete separa el contrato (especificación) de la implementación (cuerpo).
- Lo que no está en la especificación es privado: ese es el encapsulamiento.
- Las variables de paquete viven por sesión y no se revierten con ROLLBACK.
- Sobrecarga y catálogo de excepciones convierten el paquete en la API del módulo.
- Cambiar el cuerpo no invalida a nadie; cambiar la especificación sí puede.

En el laboratorio: PLSQL_03_Paquetes_pruebas.sql construye pkg_inventario, lo verifica con las pruebas P00 a P17 y cierra con cinco retos.